

WHEN SHOULD A VISION SPECIALIST DEFER? BUDGETED COLLABORATION WITH VISION-LANGUAGE MODELS

Sihang Cai¹ Rongjunchen Zhang¹ Wenxu Jia² Yipu Gong² Tao Jin²

¹HiThink Research

²Zhejiang University

ABSTRACT

When should a vision specialist call a general vision-language model (VLM)? A specialist’s low confidence does not tell us whether the VLM will give a better answer. We study this decision across 4 visual tasks and 16 model pairs. Our Learned Routing (LR) method uses the input and the specialist’s outputs to estimate whether replacing its answer will help or hurt. A normalized score ranks inputs for VLM calls, with its normalization strength and threshold chosen on validation data. We evaluate performance over the full 0–100% call range. At validation-selected thresholds, LR exceeds both standalone models in 10 pairs, using means over five training seeds. We also compare different training objectives and ways of constructing the ranking score. In some tasks, changing the score without retraining has a larger effect than changing among the tested objectives. The best score varies across tasks, and some model pairs benefit little from collaboration. Code: <https://github.com/nohi191212/ModelCollaboration>.

Index Terms— Model collaboration, visual recognition, learning to defer, budgeted inference

1. INTRODUCTION

Task-specific vision models offer efficient predictions, while general vision-language models (VLMs) can handle a broader range of inputs at greater computational cost. Their strengths need not coincide: a specialist benefits from task-specific training, whereas a VLM may resolve examples that require broader visual or semantic knowledge. This raises a practical question: when is a VLM call worth its cost? Calling it selectively may improve accuracy without paying for every input, a motivation also reflected in prompt-routing services [1].

We study this question for visual tasks in which a specialist first predicts a class, a set of boxes or an event label. A router decides whether to keep that answer or replace it with the VLM’s answer. Low specialist confidence is a natural reason to seek another opinion, but it does not tell us whether that opinion will be better. Both models may fail on the same example, and replacement can overturn an answer that was already correct. Conversely, even a VLM with lower average accuracy can help on examples that match its strengths. The value of collaboration therefore depends on which errors the models share and which they can correct for each other.

A limited call budget makes this an ordering problem: the router must rank inputs by the value of replacing their answers. As the budget grows, the chosen set expands beyond the most promising inputs to examples with smaller or uncertain gains. A useful evaluation should therefore examine the full performance–call curve. It also matters how a trained router turns its outputs into a ranking, as training supervision alone does not determine that ordering.

Prior work provides several starting points. Confidence estimation [2] and selective prediction [3] identify uncertain inputs, while learning to defer accounts for an imperfect expert [4, 5]. Its joint-classification formulations learn class outputs alongside deferral and need adaptation for box or event outputs. Post-hoc deferral [6] keeps an existing predictor fixed, making it a close comparison for answer replacement. We adapt its loss to our router inputs and backbone. Text and multimodal routing [7, 8, 9, 10, 11, 12, 13] study related choices across other model interfaces and tasks. Speculative decoding [14, 15] instead verifies draft tokens within generation.

We introduce **Learned Routing (LR)**, a compact router using input features and the specialist’s representations, predictions and confidence. Paired answers show when replacement helps or hurts. Inference uses information available before the VLM call. We compare training objectives and ranking scores separately, and use a normalized estimate of replacement gain as the default score. Our contributions are:

- (1) **Unified evaluation.** Across four tasks and 16 model pairs, we explore different training objectives and ranking scores within the LR framework, identifying where collaboration helps and where it offers little benefit.
- (2) **Method and finding.** We develop LR with normalized scoring and show that, on some tasks, score construction affects performance more than the tested training objectives.
- (3) **Open-source release.** We release all experiment code and per-example data, the 80 main-table router checkpoints, and reproduction materials.

2. PRELIMINARY

Let S be a specialist and L a VLM. For input x , the specialist first gives $S(x)$. A router then uses information $\mathcal{I}(x)$ available before calling L to choose the final answer:

$$\hat{y}(x) = \begin{cases} S(x), & a(x) = 0, \\ L(x), & a(x) = 1, \end{cases} \quad a(x) \in \{0, 1\}. \quad (1)$$

Here $a(x) = 0$ keeps the specialist answer, while $a(x) = 1$ calls the VLM and uses its answer instead. For n inputs, $\sum_i a(x_i)$ is the number of VLM calls. Given an allowed call fraction b , we seek the highest task score M :

$$\max_a M(\hat{\mathbf{y}}, \mathbf{y}) \quad \text{s.t.} \quad \sum_i a(x_i) \leq \lfloor bn \rfloor. \quad (2)$$

Here $\hat{\mathbf{y}}$ and $\mathbf{y}$ collect the final answers and ground-truth labels. For example, $b = 0.25$ allows calls on at most a quarter of the inputs. Each input counts once, even if it contains two images or multiple target objects. Our router makes this choice using a score threshold chosen on validation data. The same threshold is used on test inputs, where the fraction of calls may differ from the validation budget.

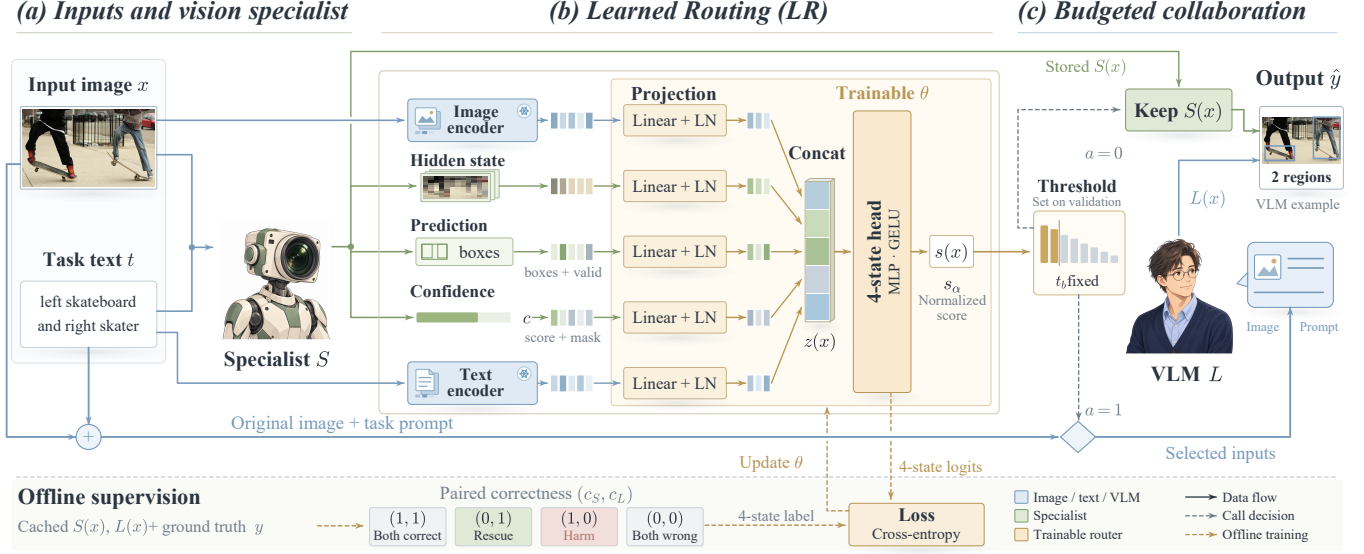


Fig. 1. Deciding when to call the VLM. After the specialist predicts, LR combines input and specialist features. The illustrated four-state variant estimates paired outcomes and normalizes its score before a validation-selected threshold decides whether to retain the specialist answer or call the VLM. Image-only tasks omit text.

3. LEARNED ROUTING

3.1. Replacement Outcomes and Gain

Let $c_S, c_L \in \{0, 1\}$ indicate specialist and VLM correctness. Their paired outcomes are both correct $(1, 1)$, rescue $R = (0, 1)$, harm $H = (1, 0)$ and both wrong $B = (0, 0)$. For delegated inputs A ,

$$\Delta \text{Acc} = \frac{|R \cap A| - |H \cap A|}{n}. \quad (3)$$

Given pre-call information $\mathcal{I}$, expected correctness gain is

$$\eta(\mathcal{I}) = \Pr(R | \mathcal{I}) - \Pr(H | \mathcal{I}). \quad (4)$$

With true conditional probabilities and equal call costs, choosing the largest positive gains maximizes expected accuracy under a budget ceiling. Estimated probabilities can yield different rankings under alternative scores. Macro-F1 is not additive, so construction performance is computed from the combined predictions rather than this gain identity.

3.2. Shared Inputs and Supervision

LR uses an 8.07M-parameter image/text backbone distilled from SigLIP2 [16] on training inputs. Distillation matches normalized image and text features separately, without task labels. The backbone and specialist remain frozen during router training.

Figure 1 combines image features, pooled specialist states, prediction summaries and confidence with missing-value indicators. Task text adds a branch on gRefCOCO and NLVR2. Each branch is projected to 128 dimensions and layer-normalized. Concatenated branches enter an MLP with 128 hidden units and GELU. Each pair learns separate weights under shared settings.

Our four-state version, LR_{4S} , uses one target state per example, or fractions over four construction events. We minimize

$$\mathcal{L}_{4S} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^4 q_{ik} \log p_{ik}, \quad p_i = \text{softmax}(\ell_i). \quad (5)$$

Here q_i is the observed state distribution. Four outputs support outcome inspection and several scoring rules. Separating both-correct from both-wrong cases need not improve ranking. We use uniform sampling without class weights or label smoothing.

For the post-hoc comparison LR_{PH} , a scalar head r on the same inputs uses the deferral loss [6]

$$\mathcal{L}_{PH} = e_S \log(1 + e^{-r}) + e_L \log(1 + e^r), \quad (6)$$

Here c_S and c_L equal 1 for correct specialist and VLM answers, respectively, and 0 otherwise. Errors are $e_S = 1 - c_S$ and $e_L = 1 - c_L$, averaged over construction events. Added call cost is zero. Sweeping the threshold varies the budget.

3.3. From Outcome Estimates to a Ranking

Our default LR_{4S} uses

$$s_\alpha(x) = \frac{p_R - p_H}{(p_R + p_H + 2p_B + \epsilon)^\alpha}, \quad \epsilon = 10^{-8}. \quad (7)$$

Here p_R, p_H and p_B predict the probabilities that replacement fixes an error, causes an error, or leaves both models wrong. At $\alpha = 0$ the score is $p_R - p_H$. Ignoring ϵ , $\alpha = 1$ ranks like $(p_R + p_B)/(p_H + p_B)$. Intermediate values interpolate. This empirical rule uses existing outputs and does not inherit true-gain optimality.

3.4. Full-Budget Selection

For a score, let $M(u_k)$ be validation performance after delegating its top $\lfloor kn/100 \rfloor$ inputs, with $u_k = \lfloor kn/100 \rfloor / n$. We evaluate the full curve,

$$\mathcal{A}_{100} = \sum_{k=1}^{100} \frac{M(u_{k-1}) + M(u_k)}{2} (u_k - u_{k-1}). \quad (8)$$

Table 1. Test results at validation-selected thresholds over 0–100% budgets. Scores and actual calls are percentages. LR is normalized LR_{4S}, averaged over five seeds. Parentheses show LR-minus-confidence changes in percentage points (green: better, red: worse). Bold marks the higher routed score. † exceeds both endpoints. M: MiniCPM-V-4.5, Q: Qwen3.8-27B-FP8.

Dataset	Specialist	VLM	Standalone		Confidence		LR	
			Spec.	VLM	Score↑	Calls↓	Score↑	Calls↓
CUB-200-2011	ResNet	M	81.43	26.56	81.43	0.00	81.43 (=0.00)	0.59 (↑ 0.59)
	ResNet	Q	81.43	54.82	81.43	0.00	81.43 (=0.00)	0.00 (=0.00)
	GLSim	M	90.16	26.56	90.16	0.00	90.13 (↓ 0.03)	0.85 (↑ 0.85)
	GLSim	Q	90.16	54.82	90.16	0.00	90.16 (=0.00)	0.00 (=0.00)
gRefCOCO	GroundingDINO	M	26.69	50.01	50.01	100.00	50.23 † (↑ 0.22)	58.19 (↓ 41.81)
	GroundingDINO	Q	26.69	74.47	74.47	100.00	75.12 † (↑ 0.65)	88.47 (↓ 11.53)
	InstanceVG	M	66.95	50.01	61.88	52.09	67.91 † (↑ 6.04)	19.04 (↓ 33.06)
	InstanceVG	Q	66.95	74.47	72.43	68.39	76.24 † (↑ 3.81)	62.00 (↓ 6.38)
NLVR2	BEiT3	M	80.00	90.74	90.77	85.80	90.80 † (↑ 0.03)	91.10 (↑ 5.30)
	BEiT3	Q	80.00	90.06	90.24	85.80	90.60 † (↑ 0.36)	84.34 (↓ 1.46)
	ViLT	M	76.42	90.74	90.64	95.93	90.69 (↑ 0.05)	98.06 (↑ 2.13)
	ViLT	Q	76.42	90.06	89.96	95.93	90.07 † (↑ 0.10)	99.62 (↑ 3.69)
ConstructionSite	YOLO26x	M	18.13	22.36	25.23	92.54	25.88 † (↑ 0.65)	45.32 (↓ 47.22)
	YOLO26x	Q	18.13	27.00	25.97	92.21	29.80 † (↑ 3.82)	44.77 (↓ 47.44)
	RT-DETR-X	M	7.89	22.36	22.36	100.00	22.80 † (↑ 0.44)	78.44 (↓ 21.56)
	RT-DETR-X	Q	7.89	27.00	27.00	100.00	26.78 (↓ 0.22)	83.57 (↓ 16.43)

Each epoch evaluates $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$. Validation $\mathcal{A}_{100}$ jointly selects the checkpoint and α . Ties favor smaller α within an epoch and earlier epochs across training. Early stopping uses the same criterion. Cross-entropy itself is unchanged. Table 1 abbreviates normalized LR_{4S} as LR.

To select a threshold, validation budgets range from 0 to 100% in 1% steps. At each interior budget, a cutoff includes tied scores only if all fit the quota. The highest validation task score selects the threshold. Ties favor fewer calls and then a smaller budget. Endpoints mean never or always calling the VLM.

The cutoff and comparison operator transfer unchanged to test inputs. Actual test calls can differ from the nominal validation budget. Confidence uses the same threshold search with negative maximum class probability or detection confidence, prioritizing missing confidence. Test labels select neither scores nor thresholds.

4. EXPERIMENTS

4.1. Datasets and Metrics

Table 2. Datasets, tasks and dataset split sizes.

Dataset	Task	Train	Val.	Test
CUB-200-2011	Fine-grained classification	4,792	1,200	5,794
gRefCOCO	Semantic grounding	209,344	14,229	35,263
NLVR2	Visual reasoning	85,042	6,982	13,937
ConstructionSite	Event detection	6,308	701	3,004

CUB [17] evaluates bird recognition and NLVR2 [18] evaluates two-image reasoning. Both use accuracy. Table 2 counts images for CUB and ConstructionSite, expressions for gRefCOCO, and paired-image statements for NLVR2.

gRefCOCO [19] evaluates whole-expression box correctness. After filtering boxes with scores below 0.7, greedy matching at generalized IoU ≥ 0.5 must leave no missed or extra instances. No-

target expressions require valid empty outputs. This differs from official segmentation evaluation.

ConstructionSite [20] uses macro-F1 across personal-protection, fall-protection, unprotected-edge and excavator-proximity violations, with one routing decision per image.

4.2. Implementation and Comparisons

Specialists are ResNet-50 [21], GLSim [22], GroundingDINO [23], InstanceVG [24], BEiT3 [25], ViLT [26], YOLO26X [27] and RT-DETR-X [28]. Each pairs with MiniCPM-V-4.5 [29] (M) and Qwen3.8-27B-FP8 [30] (Q).

LR uses AdamW, learning rate 0.003, weight decay 0.01, batch 256, at most 50 epochs and patience 8. Specialist states are taken at 90% depth. We retain the architecture from an earlier eighteen-setting search and use seeds 2026–2030. Validation selects α per pair and seed: 0, 0.25, 0.5, 0.75, 1 occur in 29, 8, 9, 9 and 25 runs, respectively. The supplement lists all choices.

Each VLM receives one temperature-zero request per example. Invalid responses remain errors and are never repaired. The supplement reports per-seed results, full curves and additional training comparisons. These analyses reuse previously inspected test sets and are exploratory, rather than a new blind evaluation.

4.3. Main Results Analysis

Table 1 shows pair-dependent benefits: useful delegation recovers specialist errors while avoiding harmful replacements.

For bird recognition, the tested VLMs rarely justify replacement, so retaining the specialist answer is the practical default. Grounding offers clearer complementarity, especially with InstanceVG. Visual reasoning favors extensive VLM use, limiting call savings. Construction benefits vary: YOLO gains from selective delegation, whereas RT-DETR-X/Q does not improve on its VLM endpoint. Comparisons should include both standalone models and the full curve. Collaboration need not beat either standalone model.

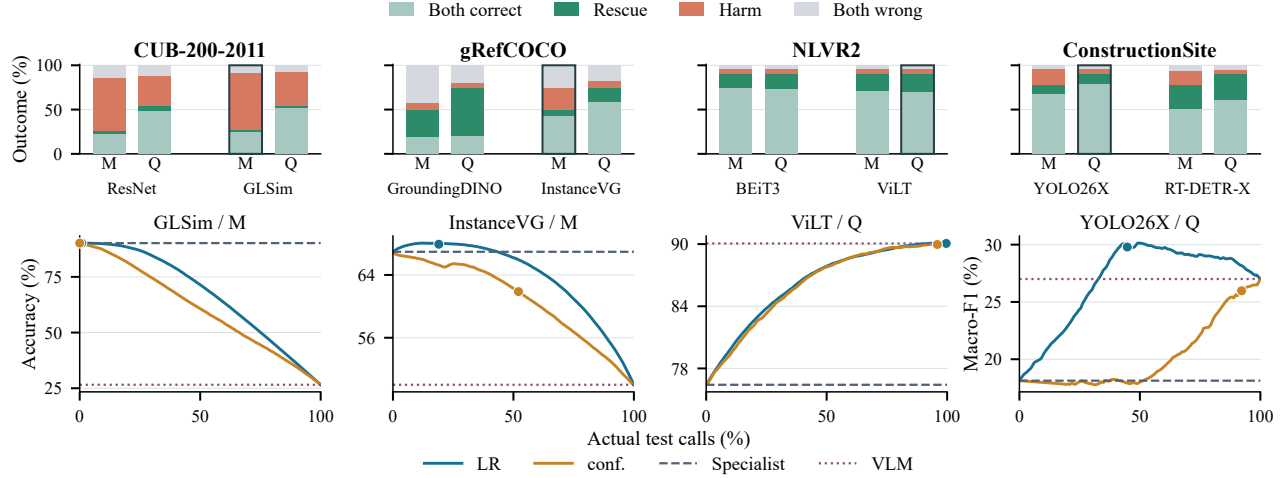


Fig. 2. Collaboration gains on test data. Top: paired outcomes for all 16 pairs, with outlines marking the pairs below. Construction counts events, other tasks examples. Bottom: performance versus actual calls, including both endpoints. LR lines average five seeds. Dots show means at independently selected validation thresholds.

4.4. Where Collaboration Gains Come From

Figure 2 relates paired outcomes to test-call curves. With the selected thresholds, InstanceVG/M corrects 727.6 expressions but introduces 388.2 errors on average, adding 0.96 accuracy points. GLSim/M has only 106 rescuable images against 3,791 harmful replacements, favoring few calls. Construction has 96.37% negative event labels. Correct negatives contribute to outcome shares but not macro-F1.

We also compare at identical actual calls. After normalization, full-budget checkpoint reselection changes task-average curves by just +0.065, +0.068, +0.102 and −0.010 points, respectively. The larger scoring effects below are therefore not explained solely by low-budget checkpoint selection.

4.5. Ablations on Supervision and Scoring

Table 3. Full-budget means (%). Avg.: equal-task mean (five-seed SD). Δ : gain over LR_{PH} (pp). Shared weights for LR_{4S} . Earlier selection settings in supplement. CS: ConstructionSite.

Variant	Score	CUB	gRef	NLVR	CS	Avg.	Δ
LR_{PH} r		65.86	59.37	87.33	21.24	58.45 (0.07)	–
3-state $p_R - p_H$		71.27	60.13	86.34	21.77	59.88 (0.06)	+1.43
Gain $\hat{\Delta}$		71.01	59.97	86.29	21.61	59.72 (0.04)	+1.27
LR_{4S} $p_R - p_H$		71.30	60.10	86.43	21.69	59.88 (0.06)	+1.43
LR_{4S} $\log(p_R/p_H)$		67.80	59.84	87.36	21.11	59.03 (0.09)	+0.58
LR_{4S} r_{4S}		67.63	59.90	87.36	21.30	59.05 (0.10)	+0.60
LR_{4S} s_α		71.37	60.66	87.36	21.63	60.25 (0.06)	+1.81

Table 3 compares variants and scores, each detailed in the supplement. Merging unchanged outcomes or predicting gain directly gives curves close to the four-state probability difference.

Changing only the score has a larger effect on some tasks. Here $r_{4S} = \log[(p_R + p_B)/(p_H + p_B)]$ estimates the optimal score of the post-hoc loss [6], whereas LR_{PH} learns its score directly. On NLVR2, this change largely closes the middle-to-high-budget gap to post-hoc deferral, but it hurts CUB. The normalized score has the highest task average, while no score is best on every task. Thus,

ranking can improve even when supervision and learned predictions remain unchanged.

4.6. Computational Cost

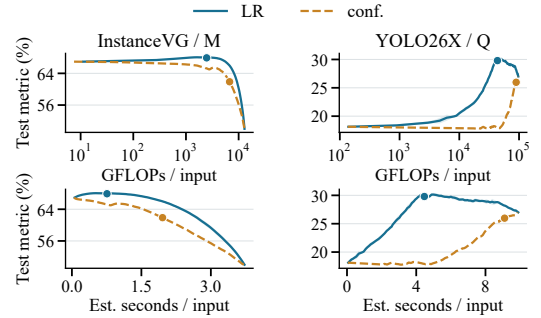


Fig. 3. Cost-performance trade-offs. InstanceVG/M (left), YOLO26X/Q (right). LR curves show five-seed means with one-SD bands. Dots average results at validation-selected thresholds. Costs are component-based estimates.

We add specialist and router costs per input, plus VLM cost at the actual call rate. Confidence omits router overhead. Component runtimes were measured on a single NVIDIA A800 GPU. These measurements estimate serial cost rather than integrated latency.

InstanceVG/M gains 6.04 points over confidence with 63.3% fewer estimated FLOPs and 61.3% less time. YOLO26X/Q gains 3.82 points with reductions of 51.4% and 50.9%.

5. CONCLUSION

Learned Routing separates estimating replacement outcomes from ranking inputs for delegation. In some tested regimes, scoring affects performance more than the choice among gain-related targets. Validation-selected normalization offers a practical compromise, and allowing zero calls avoids unnecessary replacements. Benefits remain pair-dependent: neither four-state supervision nor a fixed ratio is universally superior.

COMPLIANCE WITH ETHICAL STANDARDS

This study did not require ethical approval. The authors declare no actual or potential conflicts of interest.

6. REFERENCES

- [1] Amazon Web Services, “Understanding intelligent prompt routing in Amazon Bedrock,” Product documentation, <https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-routing.html>, Accessed September 10, 2026.
- [2] Chuan Guo, Geoff Pleiss, Yu Sun, et al., “On calibration of modern neural networks,” in *Proc. ICML*, 2017, pp. 1321–1330.
- [3] Yonatan Geifman and Ran El-Yaniv, “SelectiveNet: A deep neural network with an integrated reject option,” in *Proc. ICML*, 2019, pp. 2151–2159.
- [4] Hussein Mozannar and David Sontag, “Consistent estimators for learning to defer to an expert,” in *Proc. ICML*, 2020, pp. 7076–7087.
- [5] Rajeev Verma and Eric Nalisnick, “Calibrated learning to defer with one-vs-all classifiers,” in *Proc. ICML*, 2022, vol. 162, pp. 22184–22202.
- [6] Harikrishna Narasimhan, Wittawat Jitkittum, Aditya K. Menon, Ankit Rawat, and Sanjiv Kumar, “Post-hoc estimators for learning to defer to an expert,” in *Proc. NeurIPS*, 2022, vol. 35, pp. 29292–29304.
- [7] Lingjiao Chen, Matei Zaharia, and James Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” *arXiv preprint arXiv:2305.05176*, 2023.
- [8] Isaac Ong, Amjad Almahairi, Vincent Wu, et al., “RouteLLM: Learning to route LLMs with preference data,” *arXiv preprint arXiv:2406.18665*, 2024.
- [9] Jasper Dekoninck, Maximilian Baader, and Martin Vechev, “A unified approach to routing and cascading for LLMs,” *arXiv preprint arXiv:2410.10347*, 2024.
- [10] Zhehao Huang, Baijiong Lin, Jingyuan Zhang, Jingying Wang, Yuhang Liu, Ning Lu, Tao Li, and Xiaolin Huang, “VL-RouterBench: A benchmark for vision–language model routing,” *arXiv preprint arXiv:2512.23562*, 2025.
- [11] Haoxuan Ma, Guannan Lai, and Han-Jia Ye, “MMR-Bench: A comprehensive benchmark for multimodal LLM routing,” *arXiv preprint arXiv:2601.17814*, 2026.
- [12] Yaopei Zeng, Congchao Wang, Blake JianHeng Chen, and Lu Lin, “ReLope: KL-regularized LoRA probes for multimodal LLM routing,” *arXiv preprint arXiv:2603.24787*, 2026.
- [13] Han Wang, Deyi Ji, Lanyun Zhu, Jiebo Luo, and Roy Ka-Wei Lee, “StreamSense: Streaming social task detection with selective vision–language model routing,” in *Proc. ACM Web Conf.*, 2026.
- [14] Yaniv Leviathan, Matan Kalman, and Yossi Matias, “Fast inference from Transformers via speculative decoding,” in *Proc. ICML*, 2023, vol. 202 of *Proceedings of Machine Learning Research*, pp. 19274–19286, PMLR.
- [15] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper, “Accelerating large language model decoding with speculative sampling,” *arXiv preprint arXiv:2302.01318*, 2023.
- [16] Michael Tschanen, Alexey Gritsenko, Xiao Wang, et al., “SigLIP 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features,” *arXiv preprint arXiv:2502.14786*, 2025.
- [17] C. Wah, S. Branson, P. Welinder, et al., “The Caltech-UCSD Birds-200-2011 dataset,” Tech. Rep. CNS-TR-2011-001, California Institute of Technology, 2011.
- [18] Alane Suhr, Stephanie Zhou, Ally Zhang, et al., “A corpus for reasoning about natural language grounded in photographs,” in *Proc. ACL*, 2019, pp. 6418–6428.
- [19] Chang Liu, Henghui Ding, and Xudong Jiang, “GRES: Generalized referring expression segmentation,” in *Proc. IEEE/CVF CVPR*, 2023, pp. 23592–23601.
- [20] Xuezheng Chen and Zhengbo Zou, “Are large pre-trained vision language models effective construction safety inspectors,” *Data-Centric Engineering*, vol. 7, pp. e11, 2026.
- [21] Kaiming He, Xiangyu Zhang, Shaoqing Ren, et al., “Deep residual learning for image recognition,” in *Proc. IEEE/CVF CVPR*, 2016, pp. 770–778.
- [22] Edwin Arkel Rios, Min-Chun Hu, and Bo-Cheng Lai, “Global-local similarity for efficient fine-grained image recognition with vision Transformers,” in *Proc. IEEE ISCAS*, 2025, pp. 1–5.
- [23] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, et al., “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *Proc. ECCV*, 2024, pp. 38–55.
- [24] Ming Dai, Wenxuan Cheng, Jiang-Jiang Liu, et al., “Improving generalized visual grounding with instance-aware joint learning,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 48, pp. 448–465, 2026.
- [25] Wenhui Wang, Hangbo Bao, Li Dong, et al., “Image as a foreign language: BEiT pretraining for vision and vision-language tasks,” in *Proc. IEEE/CVF CVPR*, 2023, pp. 19175–19186.
- [26] Wonjae Kim, Bokyoung Son, and Ildoo Kim, “ViLT: Vision-and-language Transformer without convolution or region supervision,” in *Proc. ICML*, 2021, pp. 5583–5594.
- [27] Glenn Jocher, Jing Qiu, Mengyu Liu, et al., “Ultralytics YOLO26: Unified real-time end-to-end vision models,” *arXiv preprint arXiv:2606.03748*, 2026.
- [28] Yian Zhao, Wenyu Lv, Shangliang Xu, et al., “DETRs beat YOLOs on real-time object detection,” in *Proc. IEEE/CVF CVPR*, 2024, pp. 16965–16974.
- [29] Tianyu Yu, Zefan Wang, Chongyi Wang, et al., “MiniCPM-V 4.5: Cooking efficient MLLMs via architecture, data, and training recipe,” *arXiv preprint arXiv:2509.18154*, 2025.
- [30] Qwen Team, “Qwen3.8-27B-FP8,” Official model card, <https://huggingface.co/Qwen/Qwen3.8-27B-FP8>, 2026.